

Day 62: Basic Python Project Structure and `requirements.txt`

1. Day number

Day 62 of Python

You have already learned Python basics and started working with external packages. Today you will learn how to keep a small Python project organized.

2. Topic name

Project folders and `requirements.txt`

Main ideas:

- Project folder
 - Python files
 - Data folder
 - `requirements.txt`
 - Dependencies
-

3. Connection with yesterday

Yesterday, on **Day 61**, you learned how to:

- create a virtual environment
- activate it
- install a package using `pip`
- keep packages isolated from other projects

Today, you will organize those pieces into a simple project.

Think of it like this:

Day 61 → Prepare the project's Python environment Day 62 → Organize the project's files
--

4. Important topics

Project folder

A **project folder** is the main folder that contains everything related to one project.

For example:

<code>weather_project/</code>

Inside it, you might keep:

```
Python code
data
dependency information
```

Python files

Files ending in `.py` contain Python code.

For example:

```
main.py
```

`main.py` is often used as the file where a small program starts.

You can run it with:

```
python main.py
```

Data folder

A folder named `data` can store files your program uses.

For example:

```
data/
  names.txt
```

or:

```
data/
  scores.csv
```

You do **not** always need a data folder. We are using one today to practice organizing a project.

requirements.txt

`requirements.txt` is a simple text file containing the external Python packages required by the project.

Example:

```
requests==2.32.5
```

The exact version on your computer may be different.

Dependencies

A **dependency** is an external package your project depends on.

Suppose your program contains:

```
import requests
```

The requests package is a dependency because your program needs it to work.

5. Foundational notes

A small Python project might contain:

```
my_project/  
├── main.py  
├── requirements.txt  
└── data/
```

Each part has a different purpose.

```
my_project/
```

is the main project folder.

```
main.py
```

contains Python code.

```
data/
```

contains data used by the program.

```
requirements.txt
```

records external packages required by the project.

Keeping these separate makes your project easier to understand.

6. Python code vs project dependencies

These are related, but they are **not the same thing**.

Python code

Python code contains the instructions you wrote.

Example:

```
print("Hello from my project!")
```

This belongs inside something like:

```
main.py
```

Project dependencies

Dependencies are external packages your code needs.

For example, your Python file might use:

```
import requests
```

But you normally **do not put the requests package itself inside** `main.py`.

Instead, you record the dependency in:

```
requirements.txt
```

For example:

```
requests==2.32.5
```

So you can think of it as:

```
main.py
  ↓
What should my program do?

requirements.txt
  ↓
What external packages does my program need?
```

7. Easy example

Imagine a tiny project called:

```
hello_project
```

Its structure could be:

```
hello_project/
|
├─ main.py
├─ requirements.txt
└─ data/
```

Inside `main.py`:

```
print("My project is working!")
```

Inside `requirements.txt`, you might have an installed package such as:

```
requests==2.32.5
```

The data folder can stay empty for today's practice.

The exact package version may be different on your machine.

8. Problem statement

Create a small Python project.

Your project should contain:

- `main.py`
- a folder named `data`
- `requirements.txt`

You should also:

- activate your virtual environment
- use one package that you installed
- add that package to `requirements.txt`
- check that your project structure is correct

Do not build a large application. The goal is simply to practice **project organization**.

9. Concepts used

For this exercise, you will practice:

- folders
 - files
 - `.py` files
 - virtual environments
 - installed packages
 - dependencies
 - `requirements.txt`
 - running Python from the correct directory
-

10. Thought process

Before typing commands, think about the project in this order.

First: What is the main project folder?

```
my_project
```

Second: Where does my Python code belong?

```
main.py
```

Third: Where should project data go?

```
data/
```

Fourth: How will another person know which external packages the project needs?

```
requirements.txt
```

So your mental picture is:

```
Project
|
├── Code
├── Data
└── Dependencies
```

That simple separation is today's main idea.

11. Beginner-friendly pseudocode / steps

Think through the exercise like this:

```
START

Create a project folder

Move into the project folder

Create main.py

Create a data folder

Activate the virtual environment

Check which package is installed

Create requirements.txt

Put the installed package information
```

```
inside requirements.txt
```

Check the folder structure

```
Run main.py
```

```
END
```

At this stage, focus more on understanding **where things belong** than memorizing commands.

12. Suggested solving approach

Use a very simple developer workflow:

1. Open the terminal
2. Go to the location where you keep Python projects
3. Create the project folder
4. Enter the project folder
5. Create main.py
6. Create data/
7. Activate your virtual environment
8. Check your installed package
9. Record that dependency in requirements.txt
10. Check the files
11. Run main.py

A useful command for seeing installed packages is:

```
pip freeze
```

You may see something similar to:

```
requests==2.32.5
```

You can then record the package in:

```
requirements.txt
```

Another commonly used command is:

```
pip freeze > requirements.txt
```

This writes the installed packages from the active environment into the file.

For today's tiny exercise, ideally your environment should contain only the package or packages you actually need.

13. Easy edge cases

Missing file

You expect:

```
main.py
```

but accidentally forgot to create it.

Your project is incomplete.

Check that all required files exist.

Wrong folder

Suppose your terminal is currently in:

```
Desktop/
```

but your project is:

```
Desktop/my_project/
```

If you run commands from the wrong folder, files may be created in the wrong place.

Check your current location before creating files.

Empty requirements.txt

You open:

```
requirements.txt
```

and it contains nothing.

Possible reason:

```
No package was installed in the active virtual environment.
```

Another possibility is that you created the file but never added the dependency.

Check with:

```
pip freeze
```

Also make sure your virtual environment is activated.

14. Expected folder structure

At the end, your project should look approximately like this:

```
my_project/
|
├── main.py
├── requirements.txt
└── data/
```

You may also have a virtual-environment folder if you created it inside the project:

```
my_project/
|
├── venv/
├── data/
├── main.py
└── requirements.txt
```

Your `requirements.txt` might contain something like:

```
requests==2.32.5
```

Your version number does **not** need to match this example.

The important relationship is:

<code>main.py</code>	→ Python code
<code>data/</code>	→ project data
<code>requirements.txt</code>	→ external dependencies
<code>venv/</code>	→ isolated Python environment

15. Hint only

For the exercise, don't worry about writing a complicated `main.py`.

Start by solving the structure:

```
project/  
├── ?  
├── ?  
└── ?
```

Then activate yesterday's virtual environment and ask yourself:

```
Which pip command showed me the packages installed  
inside this environment?
```

Use that information to fill your dependency file.

Day 62 key idea

A project folder organizes your files; `requirements.txt` records the external packages the project needs.

Tomorrow, this foundation will make it much easier to work with projects containing multiple Python files.